

INTRODUCTION



CS/COE 0449
Introduction to
Systems Software

Luis Oliveira

(with content borrowed from wilkie and Vinicius Petrucci)

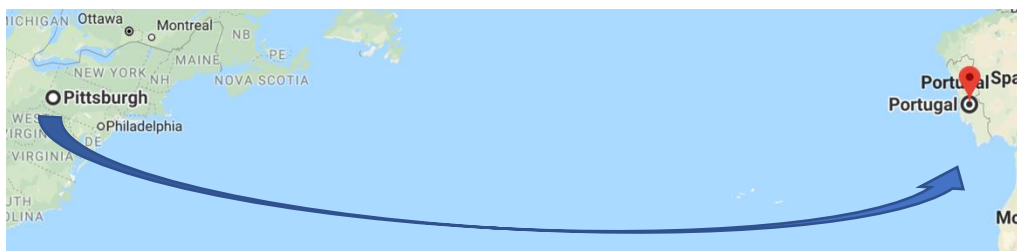
Fall 2025

SYLLABUS / ADMINISTRIVIA

I'm obligated to inform you that this is, in fact, a university course.

Welcome!

- My name is Luis (pronounced Loo-eesh, but I don't really care 😊)
- I'm not from these parts as you can tell from my accent
 - I come from Portugal



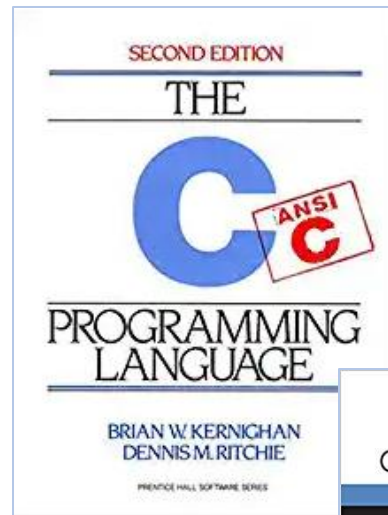
- course site: cs0449.gitlab.io/fa2025
 - all the stuff I talk about today is on the course site
- email: loliveira@pitt.edu
- office: 5421 SENSQ
- office Hours: check the site

i before e except after c...
and this guy's name!

The Textbooks

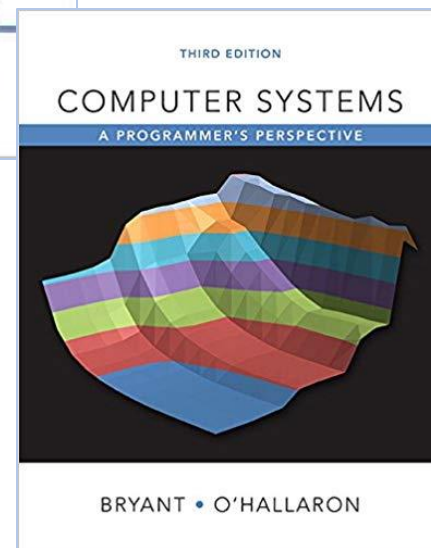
- *The ANSI C Programming Language* (2nd Edition)

- By Brian Kernighan and Dennis Ritchie
- Published by Prentice Hall, 1988
- Often called the K&R book.
- Conventions referred to as K&R style.
- Old but trustworthy!



- *Computer Systems: A Programmer's Perspective* (3rd Edition)

- By Randal E. Bryant and David R. O'Hallaron
- Published by Pearson, 2016



Course Layout

- Lectures
 - Present high-level concepts.
- Recitations
 - Applied concepts and introduce tools and skills for lab-work.
 - Clarify lectures and review topics.
 - Lab assignments!
 - Quizzes!

Policies: Grading

- I don't keep track of attendance
 - But you should come to class!
 - A lot of the concepts are best demonstrated interactively.
- Participation: 5% - Every recitation a quick quiz
- Projects: 40% - 4/5 time permitting
- Quizzes/Labs: 20% (Assigned during recitation)
- Exam 1: 15%
- Exam 2: 20%

Course Assignments

- **Programming Assignments (Projects)**
 - Roughly one-two weeks per assignment.
 - Provide deeper dive into some new skill or systems concept.
 - Programming, measurement, design.
- **Labs**
 - Problems to apply the knowledge gained in lecture
 - Can be done in groups of 2, unless explicitly stated
- **Quizzes**
 - Testing your knowledge
- **2 Exams**
 - Tests comprehension of concepts

Policies: Project Assignments

- **Collaboration**
 - Projects are an individual assignment
 - Discussion is good, sharing solutions is bad
- **Submission**
 - Electronic submission using Gradescope (no exception)
 - Check due dates on the course website (when published)
- **Linux**
 - The assignments must be performed in Linux
 - some may work in other Oses, but grading is done in Linux
- **Thoth Machine (not online yet)**
 - A machine in the CS dept, running Linux.
 - Use this machine to solve the labs (or at least test there frequently):
 - ssh_username@thoth.cs.pitt.edu
 - Use your Pitt username and password.
 - Talk to your TA if you have any issues. (Do NOT start assignments late!)

Policies: Project Checkoffs

- Checkoff meetings – randomly + suspicious submissions
 - Short meeting with course graders to validate your submission
 - You must meet within 10 days of the request
 - If you submit your project and don't checkoff: **you get a zero!**
 - If you checkoff and fail. You will do a second checkoff with me.
 - No less than 24h after the first checkoff, and last change you have
- What if you get sick?
 - Let the grader and I know about it before the checkoff deadline
 - Send e-mail to both
 - We will accommodate it as possible
 - If you communicate any issues after the checkoff deadline: **you get a zero!**
 - If you request a late checkoff meeting **after the deadline: you get a zero!**

Policies: Late Work

- You get 1 Late Day per every 2 weeks of attending lecture
 - Covers most normal setbacks and life and schedule mishaps.
 - I will have a system in place within the next couple of weeks. Until then I'll assign some late days to get you started.
- When you run out...
 - Late penalty incurs a **15% penalty for each day**. (out of original 100%)
 - An assignment **cannot be submitted after the 3rd penalty day**.
 - Four days late: that's a 0.
- Emergencies
 - Major emergencies require haste communication with me and your advisor.
- **Start everything early!!**
 - **Did I say this before?**

Policies: Lab Assignments

- **Collaboration**
 - All labs can be done in a group, unless announced differently
- **Discussion is useful to solve these problems**
 - Discuss high level concepts, not implementation details.

Policies: Conduct / Academic Integrity

- **Disability Resources / Services:**

- Contact DRS **412-648-7890**; TTY: **412-383-7355**
- They will email me, and I will listen to what they tell me to do.

- **Cheating:**

- First offence: Minimum penalty is a zero in the assignment
- Second offence: Minimum penalty is an F in the course
- All cheating will be reported!
- Pro-tip: **DON'T CHEAT**. Start early. Ask appropriate staff for help.
- The syllabus online has a more thorough policy.

- **Conduct:**

- Jokes/comments about sex, gender, race, ethnicity, religion, etc are not tolerated. Includes any online spaces involved.

More Notes about Cheating

- Again, do not cheat.
- I'm not grading projects assignments, but I still look at your work.
- Ask for help (There are PLENTY of resources)
 - TAs and my own office hours
 - Undergraduate Helpdesk (CRC)
 - We want you to succeed!
- I can definitely tell when someone cheats.
And... believe it or not, I can use google!
 - It is very obvious.
 - Do not do it.
 - The University is justifiably strict about it.
- Do not publish your code until after the semester (if at all)
 - Use private git repositories if you want to share with employers

Teaching Pedagogy / Philosophy

- I will help you, but you must ask for help.
 - You are learning, it's expected you are confused at times.
 - That's why I'm here!
- Work with other students.
 - Study groups are good!
 - Companies value ability to work with others! (You won't work alone)
- No questions are dumb!
 - You ARE learning! You are not supposed to know things...
 - If you have a question, likely other people will also have it!
 - Even if you are supposed to know that, you don't... so ask!
 - So ASK QUESTIONS!
 - **DON'T STRUGGLE IN SILENCE**

Lessons learned

- I scare some students (source: OMETs)
 - Sorry :')
 - Don't be scared! I like to help!
- If I exclaim “oh god” looking at your code
 - Sorry? :D
- Regret #1 of my students:
 - I should have started earlier ;)
- Come to lectures!
 - You have access to me, we can interact
 - You can ask questions, and get the answers promptly!
 - Please... be interactive 😊
- If you are bored, let me know

Quiz: CS 447 situation

- What is your CS 447 situation?

(A) I am **not** taking CS 447 (CS minor?)

(B) I am taking CS 447 **this** semester

(C) I completed CS 447 **last** semester

(D) I completed CS 447 at least 2 semesters ago

COURSE OVERVIEW

If food were knowledge, this would be, like, our restaurant menu.

Topics (Subject to deviation)

- We're going to (tentatively) learn **SO MUCH** fun stuff!
- The C Systems Programming Language
 - This will take us roughly 5-6 weeks!
 - We'll focus on the differences from Java
 - Talk A LOT about memory
- A bit of assemble (x86-64)
 - Reading, not writing
- How programs are made and executed
 - Linking (mad-libs), ELF's (and friends), Loading
- Operating System
 - Interrupts and Syscalls
 - Signals, processes, threads, and synchronization
 - Fun stuff!

What is Systems?

The dictionary :

- <https://www.oxfordlearnersdictionaries.com/us/definition/english/system>
 - A **group of things**, pieces of equipment, etc. that are connected or **work together**
 - A **set** of computer **equipment** and programs that are **used together**
 - Amongst others
- <https://www.merriam-webster.com/dictionary/system>
 - a regularly interacting or interdependent **group of items forming a unified whole**
 - Amongst others

What is Systems?

- **Systems is broad**
 - A subfield of CS dealing with the interactions between software/hardware.
 - A layer that provides abstractions and must constantly reevaluate them.
 - Operating Systems
 - File Systems
 - Program Analysis / Debugging Tools
 - Intra/Inter System Protocols
 - A house built from trade-offs in approach...
 - Do you build better hardware? Add more memory?
 - Or, do you design better software?
 - And trade-offs in design...
 - Do you choose the specialized path?
 - Or, do you create a general system?
 - Both??
 - Very opinionated!!!!!!!!!!!!

Why the C Programming Language?

- Because B sucks and D wasn't invented yet. J/K.
- C was invented in 1972 alongside UNIX to an effort to aid application development of that system.
- Eventually UNIX itself was rewritten in C cementing C as a systems language.
- As such, C provides a high-level abstraction of assembly / machine-code and a low-level abstraction of memory, from the perspective of the C programmer.
 - This is important for programming systems code!
 - Allows full manipulation of memory (to one's peril, often.)
 - This, in turn, allows for full manipulation of cpu/hardware.

Why the C Programming Language??

- Learning C helps you understand Systems.
- Understanding Systems lets you make them better.
 - Or break them. 😊
- C reveals the underlying memory model and execution environment.
 - Lets you understand **any** program.
 - Even if you do not have the original code.
- Failing at C helps you learn...
 - Because then you debug your program.
 - And debuggers are very useful tools.



How people use these skills

- **Writing Operating Systems**
 - not the entire thing hopefully
 - ... but parts are generally gonna be C/C-like
- Understanding systems means knowing how to mitigate/improve performance.
 - Important that your abstractions don't hurt performance because EVERY user application suffers.
 - Yet, performance is not the only consideration; understanding abstractions should help alleviate design fatigue. <https://wilkie.how/posts/kaashoeks-law>
- Linux and Device Drivers: 10+ million lines of C
 - Yikes.
 - But, learning C means you can potentially read this and learn more about / improve / extend Linux.



How people use these skills

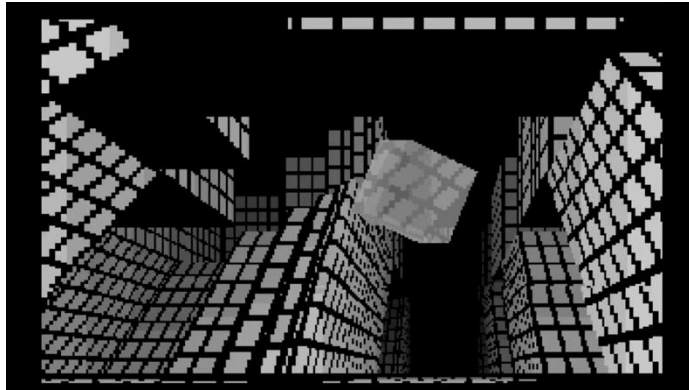
- **Debugging Higher-level Programs**
 - Yes, even Python itself crashes!
 - ... and the Python interpreter is written in C ...
 - ... and computers don't understand C ...
 - ... so it's gonna give you an assembly dump.



How people use these skills

- **Creating Art**

- Real-time art includes not just video games.
- There is a lot of fun and skill involved.
- Being creative within a constraint has been very alluring.
- The Demoscene is such a community.



How people use these skills

- **Breaking Things for Great Good**
 - Or great bad... I'm not your parents.
 - Why? Old programs with copy-protection are still useful.
 - Original source code backed up??
What time do you think this is?? Never????
 - And it is technically legal to reverse-engineer and/or change them.
 - The best kind of legal.
 - But I'm not a lawyer and this is not legal advice. lol
 - You will typically use a “debugger” to break down a program's behavior.
 - And then patch it to do / not-do things.
 - Generally done professionally by librarians/archivists.
 - We will also do this!!



How YOU will use these skills

- All of the above!!
 - Some in this course ;)
- And, of course, **TO HAVE FUN!!**